```
# 1. Print using sep and end
print("Python", "Fundamentals", "Test", sep="|", end="!\n")

# 2. Multiple assignment and escape characters
name, city, year = "Asha", "Surat", 2026
print("Name:\t", name, "\nCity:\t", city)

# 3. Print both types of quotes using variable
print("She said \"It's", year, "\"", sep="")

# 4. Swap values without temporary variable
a, b = 10, 20
a, b = b, a

print("a =", a)
print("b =", b)


# 1. Take input from the user
item = input("Enter item name: ")
price = float(input("Enter price: "))

raw_qty = input("Enter quantity: ")

print(type(raw_qty))

quantity = int(raw_qty)

print(type(quantity))


# 2. Calculate total using f-string
total = price * quantity

print(f"Total for {item}: {total:.2f}")


# 3. Show errors safely

try:
    int("25.5")
except ValueError:
    print("ValueError")

try:
    "25" + 5
except TypeError:
    print("TypeError")


# 4. Type conversion and floor division
print(int(float("25.5")), int(-9.9), -9.9 // 1)

# 1. List of type names
vals = [42, 3.5, 'Python', True, None]
print([type(v).__name__ for v in vals])

# 2. Dynamic typing
```

```
x = 10
x = 'ten'
print(type(x).__name__)

# 3. Floating-point comparison
print(0.1 + 0.2 == 0.3, round(0.1 + 0.2, 2))

# 4. Boolean comparison and addition
print(True == 1, True + True)

# 5. Boolean values of different objects
print([bool(v) for v in [0, '', None, [], 'False', 5]])

# 6. Check using isinstance
print(isinstance(25, (int, float)))
```

sectionA.py | Console output screenshot

Console output - sectionA.py

```
Python|Fundamentals|Test!
Name:    Asha
City:    Surat
She said "It's2026"
a = 20
b = 10
Enter item name: Enter price: Enter quantity: <class 'str'>
<class 'int'>
Total for Notebook: 399.00
ValueError
TypeError
25 -9 -10.0
['int', 'float', 'str', 'bool', 'NoneType']
str
False 0.3
True 2
[False, False, False, False, True, True]
True
```

```
# 1. Floor division and modulus
a, b = 17, 5
print(a // b, a % b, -a // b, -a % b)

# 2. Floor division rule
print((-a // b) * b + (-a % b) == -a)

# 3. Operator precedence
print(2 + 3 * 4 ** 2 // 5, 2 ** 3 ** 2, 7 // -2)

# 4. Type of division
print(type(10 / 2).__name__, type(10 // 2).__name__)

# 5. Shopping bill
price = 250
quantity = 3
tax_rate = 18

subtotal = price * quantity
tax = subtotal * tax_rate / 100
total = subtotal + tax

print(f"Subtotal: {subtotal}  Tax: {tax:.2f}  Total: {total:.2f}")

# 1. Chained comparison
age = 25
has_id = True
has_guardian = False

print(18 <= age <= 60)

# 2. Logical AND and OR
can_enter = (age >= 18 and has_id) or has_guardian
print(can_enter)

# 3. Logical operators
print("Hello" and "World", "" or "Guest", not 10)

# 4. Short-circuit evaluation
user = ""
print(user or "Guest")

# 5. Compound assignment operators
score = 10

score += 5
score *= 2
score //= 4
score **= 2

print(score)
```

sectionB.py | Console output screenshot

Console output - sectionB.py

```
3 2 -4 3
True
11 512 -4
float int
Subtotal: 750  Tax: 135.00  Total: 885.00
True
True
World Guest False
Guest
49
```

```
# Create a string
word = "Fundamentals"

# 1. First character, last character and length
print(word[0], word[-1], len(word))

# 2. String slicing
print(word[:3], word[3:], word[::-1], word[::2])

# 3. Try changing a character (TypeError)
try:
    word[0] = "X"
except TypeError:
    print("TypeError")

# Try accessing an invalid index (IndexError)
try:
    print(word[50])
except IndexError:
    print("IndexError")

# 4. Build a new string
new_word = "X" + word[1:]
print(new_word)

# Given messy text
raw = " pRiYa deSai , Surat , priya@mail.com "

# 1. Remove outer spaces and split the text
clean = raw.strip()

parts = [part.strip() for part in clean.split(",")]

print(parts)

# 2. Unpack parts into variables
name, city, email = parts

print(name.title(), city.upper(), email.lower(), sep="|")

# 3. Check email conditions
print(email.endswith(".com"), email.startswith("priya"),
      email.find("@"), email.find("#"))

# 4. Find invalid character using index()
try:
    email.index("#")
except ValueError:
    print("ValueError")

# 5. Replace every 'a' with '@'
print(city.replace("a", "@"))

# 6. Check lstrip() and rstrip()
print(raw.lstrip() == clean, raw.rstrip() == clean)

# Given values
customer = "Rahul"
```

```
item = "Laptop bag"
price = 1450
qty = 2
disc = 0.125

total = price * qty * (1 - disc)

# 1. Receipt using triple-quoted f-string
print(f"""
----- SHOPPING RECEIPT -----
Customer: {customer}
Item: {item}
Price: {price}
Quantity: {qty}
Discount: {disc:.1%}
Total: {total:.2f}
----------------------------
""")

# 2. Using format() with empty placeholders
print("{} bought {} item(s)".format(customer, qty))

# 3. Using format() with named placeholders
print("{name} paid {amount:.1f}".format(name=customer, amount=total))

# 4. Using old % formatting style
print("%s bought %d item(s) for %.2f" % (customer, qty, total))
```

sectionC.py | Console output screenshot

```
Console output - sectionC.py

F s 12
Fun damentals slatnemadnuF Fnaetl
TypeError
IndexError
Xundamentals
['pRiYa deSai', 'Surat', 'priya@mail.com']
Priya Desai|SURAT|priya@mail.com
True True 5 -1
ValueError
Sur@t
False False

----- SHOPPING RECEIPT -----
Customer: Rahul
Item: Laptop bag
Price: 1450
Quantity: 2
Discount: 12.5%
Total: 2537.50
----------------------------

Rahul bought 2 item(s)
Rahul paid 2537.5
Rahul bought 2 item(s) for 2537.50
```

```
# Starting list
items = ['Amit', 'Priya', 'Rahul', 'Sneha']

# 1. List methods
items.append("Vikram")
items.extend(['Neha', 'Karan'])
items.insert(1, "Zoya")
items.remove("Rahul")

# 2. Remove last item using pop()
last = items.pop()

# Change item at index 0
items[0] = "Aman"

# 3. Print list, last item and length
print(items)
print(last, len(items))

# 4. Difference between append() and extend()

list1 = ['a', 'b']
list1.append(['c', 'd'])

list2 = ['a', 'b']
list2.extend(['c', 'd'])

print(list1, len(list1))
print(list2, len(list2))

# 5. Show three errors safely

try:
    items[50]
except IndexError:
    print("IndexError")

try:
    items.remove("Nobody")
except ValueError:
    print("ValueError")

try:
    [].pop()
except IndexError:
    print("IndexError")

# Create a tuple
t = (10, 20, 20, 30, 40)

# 1. Indexing and slicing
print(t[0], t[-1], t[1:3])

# 2. Count and index methods
print(t.count(20), t.index(30))

# 3. Single value and single-item tuple
x = (5)
```

```
y = (5,)

print(type(x).__name__, type(y).__name__)

# 4. Show three errors safely

try:
    t[0] = "yellow"
except TypeError:
    print("TypeError")

try:
    t.append(99)
except AttributeError:
    print("AttributeError")

try:
    t.index(1000)
except ValueError:
    print("ValueError")

# 5. Modify list inside a tuple
data = ([1, 2, 3], 'hello')

data[0].append(4)
print(data)

# 6. Create a new tuple with first item replaced
new_t = ("yellow",) + t[1:]
print(new_t)
```

sectionD.py | Console output screenshot

Console output - sectionD.py

```
['Aman', 'Zoya', 'Priya', 'Sneha', 'Vikram', 'Neha']
Karan 6
['a', 'b', ['c', 'd']] 3
['a', 'b', 'c', 'd'] 4
IndexError
ValueError
IndexError
10 40 (20, 20)
2 3
int tuple
TypeError
AttributeError
ValueError
([1, 2, 3, 4], 'hello')
('yellow', 20, 20, 30, 40)
```

```
# 1. Create a set from a list
nums = [4, 7, 4, 9, 7, 7, 2, 9]

s = set(nums)

print(sorted(s), len(nums), len(s))

# 2. Set methods
s.add(11)
s.discard(99)
s.remove(2)

print(sorted(s))

# 3. Show KeyError safely
try:
    s.remove(1000)
except KeyError:
    print("KeyError")

# 4. Set operations
s1 = {1, 2, 3, 4, 5}
s2 = {4, 5, 6, 7}

print(sorted(s1 | s2), sorted(s1 & s2))

print(sorted(s1 - s2), sorted(s2 - s1),
      sorted(s1 ^ s2))

# 5. Type names
print(type({}).__name__, type(set()).__name__)

# 6. Membership operators
print(9 in s, 9 not in s1)

# 7. Try to create a set containing lists
try:
    {[1, 2], [3, 4]}
except TypeError:
    print("TypeError")

# Create dictionary
d = {
    'name': 'Rahul',
    'age': 20,
    'course': 'Python',
    'marks': [80, 85, 90],
    'address': {'city': 'Ahmedabad', 'country': 'India'}
}

# 1. Access dictionary values
print(d["name"], d.get("phone"), d.get("phone", "Not found"))

# 2. Update age and add grade
d["age"] = 21
d["grade"] = "A"

print(d["age"], len(d))
```

```
# 3. Remove items using pop() and del
removed = d.pop("course")

print(removed, d.pop("zzz", "none"))

del d["marks"]

print(list(d.keys()))

# 4. Access nested dictionary
print(d["address"]["city"])

# 5. Membership operators
print("name" in d, 'Rahul' in d, 'Rahul' in d.values())

# 6. Loop through dictionary items
for key, value in d.items():
    print(key, "->", value)

# 7. Handle KeyError
try:
    print(d["email"])
except KeyError:
    print("KeyError")

# 8. Duplicate keys
dup = {"name": "first", "x": 1, "name": "second"}

print(dup)
```

sectionE.py | Console output screenshot

Console output - sectionE.py

```
[2, 4, 7, 9] 8 4
[4, 7, 9, 11]
KeyError
[1, 2, 3, 4, 5, 6, 7] [4, 5]
[1, 2, 3] [6, 7] [1, 2, 3, 6, 7]
dict set
True True
TypeError
Rahul None Not found
21 6
Python none
['name', 'age', 'address', 'grade']
Ahmedabad
True False True
name -> Rahul
age -> 21
address -> {'city': 'Ahmedabad', 'country': 'India'}
grade -> A
KeyError
{'name': 'second', 'x': 1}
```

```
# Given student records
records = [
    ('Asha', 'Python', 78),
    ('Asha', 'SQL', 82),
    ('Asha', 'Excel', 71),
    ('Ravi', 'Python', 55),
    ('Ravi', 'SQL', 61),
    ('Ravi', 'Excel', 49),
    ('Meena', 'Python', 92),
    ('Meena', 'SQL', 88),
    ('Meena', 'Excel', 95),
    ('Karan', 'Python', 64),
    ('Karan', 'SQL', 70),
    ('Karan', 'Excel', 58),
]

# 1. Build a set of courses
courses = set()

for who, course, marks in records:
    courses.add(course)

print(sorted(courses))

# 2. Build dictionary of student scores
scores = {}

for who, course, marks in records:
    if who not in scores:
        scores[who] = []
    scores[who].append(marks)

# 3. Calculate average and grade
averages = {}

for who, marks in scores.items():
    avg = sum(marks) / len(marks)

    if avg >= 75:
        grade = "A"
    elif avg >= 60:
        grade = "B"
    else:
        grade = "C"

    averages[who] = avg

    print(f"{who:<8} avg={avg:.1f} grade={grade}")

# 4. Top student and overall average
top = max(averages, key=averages.get)
overall = sum(averages.values()) / len(averages)

print(f"Top: {top} | Overall average: {overall:.2f}")

# 5. Students above overall average
above = [who for who, avg in averages.items() if avg > overall]
```

```
print("Above overall:", above)

# 6. Smallest and largest marks
all_marks = [marks for who, course, marks in records]

extremes = (min(all_marks), max(all_marks))

print(extremes, type(extremes).__name__)

# 7. Why we used these data types
# Set stores unique courses, dictionary stores student scores,
# list stores multiple marks, and tuple stores fixed extremes.
```

sectionF.py | Console output screenshot

Console output - sectionF.py

```
['Excel', 'Python', 'SQL']
Asha     avg=77.0 grade=A
Ravi     avg=55.0 grade=C
Meena    avg=91.7 grade=A
Karan    avg=64.0 grade=B
Top: Meena | Overall average: 71.92
Above overall: ['Asha', 'Meena']
(49, 95) tuple
```